

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 2

Roll no:12 PRN No: 12414908 Name of Student: Srushti Patil

Subject Name : Deep Learning

Problem Statement :

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm :

1. Import the required Python libraries.
2. Load the Wine dataset from Scikit-learn.
3. Convert the dataset into a Pandas DataFrame.
4. Explore the dataset using summary statistics and check for missing values.
5. Visualize the class distribution and feature correlations.
6. Split the dataset into training and testing sets.
7. Standardize the feature values using StandardScaler.
8. Create an MLP classifier with two hidden layers.
9. Train the model using the training data.
10. Predict the class labels for the test data.
11. Evaluate the model using accuracy, confusion matrix, and classification report.
12. Visualize the confusion matrix.
13. Analyze the results and conclude the performance of the MLP classifier.

Code :

```
1.Import Library
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

from sklearn.neural_network import MLPClassifier
```

2.Load Dataset

```
wine = load_wine()

X = wine.data
y = wine.target

print("Feature Shape:", X.shape)
print("Target Shape:", y.shape)
```

3.create Dataframe

```
df = pd.DataFrame(X, columns=wine.feature_names)

df["Target"] = y

df.head()
```

4.Check dataset information

```
df.info()

df.describe()
```

5.check missing values

```
df.isnull().sum()
```

6.Visualize class distribution

```
sns.countplot(x=y)

plt.title("Wine Classes")
plt.xlabel("Class")
```

```
plt.ylabel("Count")
plt.show()
```

7. Correlation Heatmap

```
plt.figure(figsize=(12,8))

sns.heatmap(df.corr(), cmap="coolwarm")

plt.title("Correlation Heatmap")
plt.show()
```

8. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=42
)
```

9. Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

10. Build MLP Model

```
mlp = MLPClassifier(
    hidden_layer_sizes=(100,50),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)
```

```
mlp.fit(X_train, y_train)
```

11. Prediction

```
y_pred = mlp.predict(X_test)
```

12. Accuracy

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

12. Classification report

```
print(classification_report(y_test, y_pred))
```

13. Confusion matrix

```
cm = confusion_matrix(y_test, y_pred)
```

```
print(cm)
```

14. Visualize confusion matrix

```
plt.figure(figsize=(6,5))
```

```
sns.heatmap(cm,  
             annot=True,  
             fmt='d',  
             cmap='Blues',  
             xticklabels=wine.target_names,  
             yticklabels=wine.target_names)
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
```

```
plt.show()
```

Output :

```
[ ] import numpy as np  
import pandas as pd  
  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.datasets import load_wine  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report  
  
from sklearn.neural_network import MLPClassifier  
  
[ ] wine = load_wine()  
x = wine.data  
y = wine.target  
  
print("Feature Shape:", x.shape)  
print("Target Shape:", y.shape)  
... Feature Shape: (178, 13)  
Target Shape: (178,)
```

```
df = pd.DataFrame(X, columns=wine.feature_names)

df["Target"] = y

df.head()
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	
0	14.23	1.71	2.43		15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	3.92
1	13.20	1.78	2.14		11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	3.40
2	13.16	2.36	2.67		18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	3.17
3	14.37	1.95	2.50		16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	3.45
4	13.24	2.59	2.87		21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04	2.93

proline	Target
1065.0	0
1050.0	0
1185.0	0
1480.0	0
735.0	0

```
df.describe()
```

<class 'pandas.core.frame.DataFrame'>													
RangeIndex: 178 entries, 0 to 177													
Data columns (total 14 columns):													
#	Column			Non-Null Count					Dtype				
0	alcohol			178 non-null					float64				
1	malic_acid			178 non-null					float64				
2	ash			178 non-null					float64				
3	alcalinity_of_ash			178 non-null					float64				
4	magnesium			178 non-null					float64				
5	total_phenols			178 non-null					float64				
6	flavanoids			178 non-null					float64				
7	nonflavanoid_phenols			178 non-null					float64				
8	proanthocyanins			178 non-null					float64				
9	color_intensity			178 non-null					float64				
10	hue			178 non-null					float64				
11	od280/od315_of_diluted_wines			178 non-null					float64				
12	proline			178 non-null					float64				
13	Target			178 non-null					int64				
dtypes: float64(13), int64(1)													
memory usage: 19.6 KB													
	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od	
count	178.000000	178.000000	178.000000		178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	
mean	13.000618	2.336348	2.366517		19.494944	99.741573	2.295112	2.029270	0.361854	1.590899	5.058090	0.957449	
std	0.811827	1.117146	0.274344		3.339564	14.282484	0.625851	0.998859	0.124453	0.572359	2.318286	0.228572	
min	11.030000	0.740000	1.360000		10.600000	70.000000	0.980000	0.340000	0.130000	0.410000	1.280000	0.480000	
25%	12.362500	1.602500	2.210000		17.200000	88.000000	1.742500	1.205000	0.270000	1.250000	3.220000	0.782500	
50%	13.050000	1.865000	2.360000		19.500000	98.000000	2.355000	2.135000	0.340000	1.555000	4.690000	0.965000	
75%	13.677500	3.082500	2.557500		21.500000	107.000000	2.800000	2.875000	0.437500	1.950000	6.200000	1.120000	
max	14.830000	5.800000	3.230000		30.000000	162.000000	3.880000	5.080000	0.660000	3.580000	13.000000	1.710000	

df.isnull().sum()

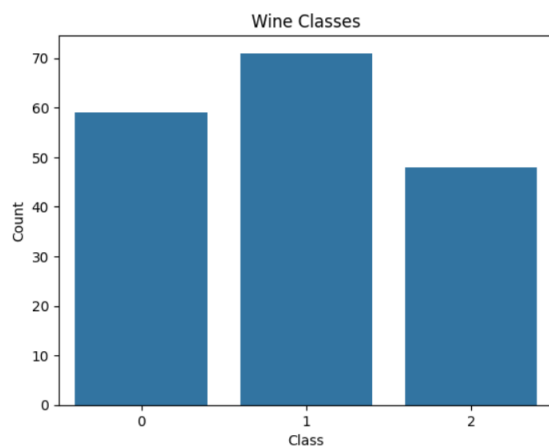
...

0

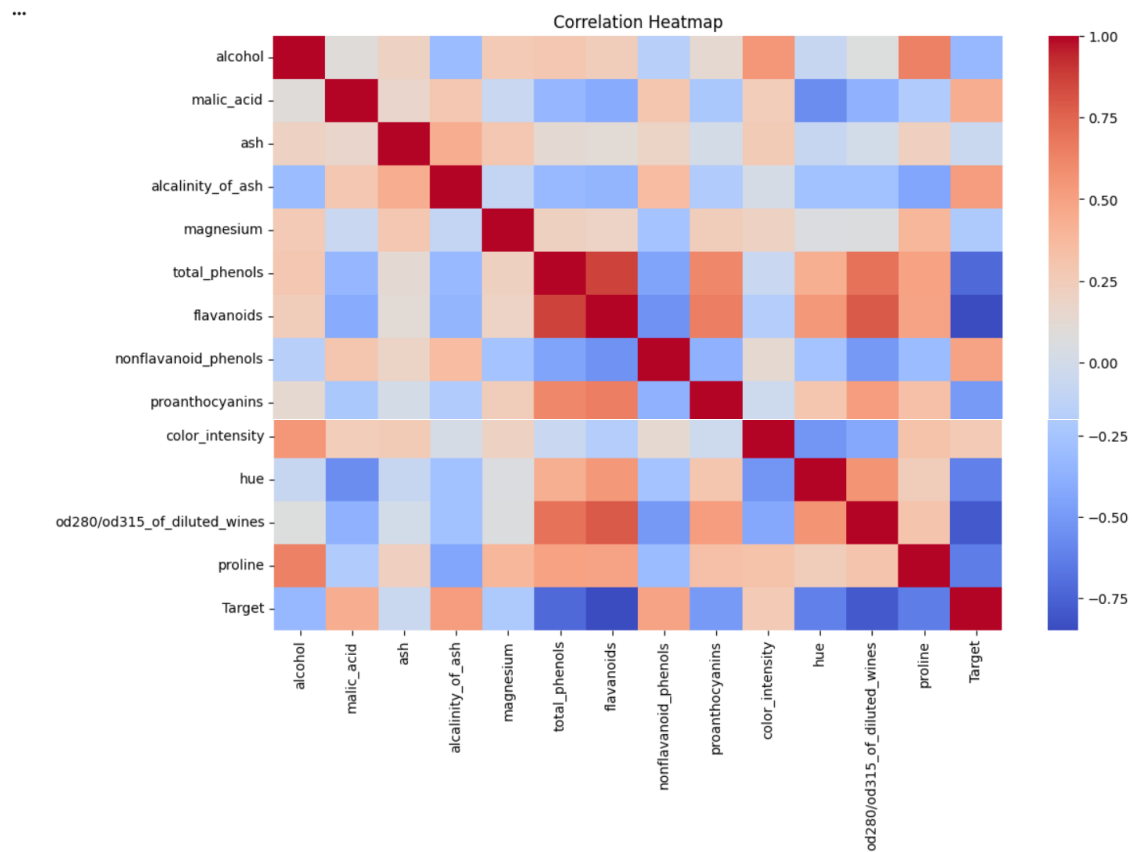
alcohol	0
malic_acid	0
ash	0
alcalinity_of_ash	0
magnesium	0
total_phenols	0
flavanoids	0
nonflavanoid_phenols	0
proanthocyanins	0
color_intensity	0
hue	0
od280/od315_of_diluted_wines	0
proline	0
Target	0

dtype: int64

```
plt.title("Wine Classes")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```



```
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
mlp = MLPClassifier(
    hidden_layer_sizes=(100,50),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)

mlp.fit(X_train, y_train)
```

...

MLPClassifier

MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=1000, random_state=42)

```
y_pred = mlp.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
Accuracy: 1.0
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	14
1	1.00	1.00	1.00	14
2	1.00	1.00	1.00	8
accuracy			1.00	36
macro avg	1.00	1.00	1.00	36
weighted avg	1.00	1.00	1.00	36

```
cm = confusion_matrix(y_test, y_pred)
```

```
print(cm)
```

```
... [[14  0  0]
      [ 0 14  0]
      [ 0  0  8]]
```

```
plt.figure(figsize=(6,5))
```

```
sns.heatmap(cm,
             annot=True,
             fmt='d',
             cmap='Blues',
             xticklabels=wine.target_names,
             yticklabels=wine.target_names)
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
```

```
plt.show()
```

